DATABASE MANAGEMENT SYSTEMS PROJECT REPORT



Resident & Maintenance Billing System

Submitted by:
Name: Krish
Roll Number: 25WU0102333
Section: AIML Rhinos

under supervision of

Prof. Sahithi

Abstract

Residential societies maintain information about flats, residents, monthly maintenance bills, payments and service complaints. When these records are handled through paper registers or separate spreadsheets, office staff can face duplicate entries, calculation errors and delays in identifying outstanding dues. This project presents a relational database design and a small working prototype for a Resident and Maintenance Billing System.

The database design contains five core entities: FLAT, RESIDENT, MAINTENANCE_BILL, PAYMENT and COMPLAINT. Primary keys identify records and foreign keys preserve relationships between flats, residents, bills, payments and complaints. The prototype provides browser-based screens for registering flats and residents, generating monthly bills, recording full or partial payments, tracking complaint status and reviewing collection reports. The locally runnable demo uses Python and SQLite to reduce setup needs, while a separate MySQL 8 schema includes constraints and payment-status triggers for the database implementation.

The prototype is intended for academic demonstration and uses fictional sample records. It is not a production service and does not include production authentication, payment-gateway integration or remote hosting.

Keywords: DBMS, relational database, primary key, foreign key, maintenance billing, payment tracking, SQL, residential society.

Acknowledgement

I would like to express my gratitude to the faculty member guiding this project and to everyone who provided feedback during the project review. Preparing this project helped me connect database concepts such as ER modelling, relational schemas, integrity constraints and SQL reporting with a practical management workflow. The faculty/supervisor name and any additional acknowledgement required by the department should be added before final printing.

Contents

Section	Description
Abstract	Project overview and approach
1	Introduction, problem statement, objectives and scope
2	Existing system, proposed system and requirements
3	Database design and relational schema
4	Prototype implementation and workflow
5	Testing and results
6	Conclusion and future scope
References	Project source material and technical documentation

Abbreviations

Term	Meaning
DBMS	Database Management System
PK	Primary Key
FK	Foreign Key
SQL	Structured Query Language
ER	Entity-Relationship
UI	User Interface

1. Introduction

A residential society office coordinates information about apartments, people living in each flat, monthly maintenance charges, money collected and service issues raised by residents. A Database Management System provides a structured way to store this information and retrieve it when required. The project focuses on the society-office workflow described in the Review 1 presentation.

1.1 Problem statement

Manual registers and disconnected spreadsheets make it difficult to maintain consistent resident records, calculate recurring bills correctly, track partial payments and quickly prepare a list of flats with outstanding dues. A centralized relational database can reduce duplicate records and support repeatable reporting.

1.2 Objectives

- Store flat, resident, maintenance bill, payment and complaint information in related tables.
- Generate monthly maintenance bills for flats using a configurable area-based calculation in the demo.
- Record payments and identify unpaid, part-paid and paid bills.
- Provide due-list and monthly collection reports using database queries.
- Demonstrate primary keys, foreign keys, uniqueness and payment-status handling.

1.3 Scope

The current scope covers core society-office records and a local browser-based demonstration. It does not implement resident-facing accounts, online payment processing, email/SMS alerts or production-grade access control. The code and MySQL schema are suitable as an academic base that can be extended after further requirements are collected.

1.4 Intended users

- Society administrator: reviews records, bills, collections and reports.
- Office staff: registers residents, enters payments and updates complaints.
- Project reviewer: checks the prototype workflow, SQL structure and report outputs.

2. Existing and Proposed System

2.1 Existing system and limitations

Existing approach in the review	Limitation described in the review
Paper registers and Excel sheets	Errors and inconsistent or duplicate records
Manual monthly bill calculations	Incorrect bill amounts or totals
Handwritten receipts	Slower payment reconciliation
Dues tracked through notes or memory	No quick defaulter list and greater risk of losing records

2.2 Proposed system

The proposed system uses a relational database as a single source of truth. Resident records are linked to flats, maintenance bills reference flats, payments reference bills and complaints reference residents. Reports are generated from stored data rather than recalculated manually each time.

2.3 Functional modules

Module	Main functions
Dashboard	Summary counts, bill status and recorded collection
Flats	Add and list wing, floor and area details
Residents	Register owners/tenants and link them to a flat
Maintenance billing	Generate bills for a selected month and year; skip duplicate billing periods
Payments	Record payments, support part-payment and update bill status
Complaints	Create service requests and update status
Reports	List outstanding dues and monthly billed-versus-collected totals

2.4 Basic requirements

- Python 3.10 or newer for the local demonstration.
- A modern desktop browser and a terminal to start the local server.
- SQLite for the runnable local demo; MySQL 8+ for executing the separate MySQL schema.
- No third-party Python packages are required for the local prototype.

3. Database Design

3.1 Entities and key attributes

Table	Primary key	Important attributes	Purpose
FLAT	flat_no	wing, floor, area_sqft	Records each flat in the society
RESIDENT	resident_id	name, phone, type, flat_no (FK)	Stores owner and tenant records
MAINTENANCE_BILL	bill_id	flat_no (FK), month, year, amount, due_date, status	Stores recurring bills
PAYMENT	payment_id	bill_id (FK), pay_date, amount_paid, mode	Stores transactions against a bill
COMPLAINT	complaint_id	resident_id (FK), category, date_raised, status	Tracks resident service requests

3.2 Relationships

- FLAT to RESIDENT: one flat can be linked to one or more resident records; each resident record references a flat.
- FLAT to MAINTENANCE_BILL: a flat can have a bill for each billing month; each bill references a flat.
- MAINTENANCE_BILL to PAYMENT: a bill can have multiple payment transactions, which supports partial payments.
- RESIDENT to COMPLAINT: a resident can raise multiple complaints; each complaint references the resident who raised it.

3.3 Integrity rules

- Primary keys uniquely identify rows in each table.
- Foreign keys link dependent rows to valid parent records and use restricted deletion to protect linked data.
- A unique constraint on (flat_no, month, year) prevents duplicate monthly bills for the same flat.
- Check constraints restrict valid month values, positive flat area and positive payment amounts.
- Bill status is recalculated from the sum of payments: UNPAID, PARTIAL or PAID.

3.4 Normalization

The design separates flat details, resident details, bills, payments and complaints into distinct relations so that one type of fact is primarily maintained in one place. For example, the flat wing and area are stored in FLAT rather than repeated in every bill. Payment transactions are kept separate from maintenance bills, which avoids overwriting previous payments when a resident pays in instalments. These choices support a normalized relational structure; the final MySQL schema should be reviewed against the course's exact normalization checklist if formal 1NF/2NF/3NF proof is required.

4. Prototype Implementation

4.1 Technology stack

Component	Role in this submission
Python standard library	HTTP server, request handling and server-side workflow logic
SQLite	Local relational storage so the demonstration can run without database-server setup
MySQL 8 SQL script	Separate schema, integrity constraints and payment-status triggers
HTML and CSS	Browser-based pages, forms, tables and dashboard layout

4.2 Running the prototype

1. Open a terminal in the repository directory.
2. Run `python run.py` using Python 3.10 or newer.
3. Open `http://127.0.0.1:8000` in the browser.
4. Use the navigation menu to register flats/residents, generate bills, record payments, update complaints and open reports.
5. Stop the local server with Ctrl+C.

On first run, the program creates `app/instance/society.db` and seeds fictional demo records. Deleting this database file while the server is stopped resets the dataset. The prototype is designed for local academic demonstration and does not include production login or remote hosting.

4.3 Monthly bill calculation

The monthly billing screen accepts the billing month, year, due date and a configurable demonstration rate. For each flat, the prototype calculates `bill amount = area_sqft × rate` and attempts to insert a bill for that period. The unique key prevents duplicates. The sample rate of ₹5 per square foot is an example input only; it was not specified in the Review 1 presentation and should be changed to the rate approved for the class demonstration, if any.

4.4 Payment status logic

Condition	Bill status
Total payment = 0	UNPAID
Total payment is greater than 0 but below bill amount	PARTIAL
Total payment reaches the bill amount	PAID

The local Python application recalculates status after a payment is recorded and prevents a payment larger than the outstanding balance. The MySQL script implements the same status recalculation pattern using triggers for payment insert, update and delete operations.

5. Testing and Results

A standard-library smoke test was run against a temporary database. It checked whether the pages and stylesheet respond and exercised the main data-entry paths without modifying the user's normal demo database.

Test	Action	Expected result	Observed result
T1	Open dashboard, flats, residents, bills, payments, complaints and reports	Pages respond successfully	PASS
T2	Add a flat through the web form	Flat is saved and shown in the directory	PASS
T3	Generate bills for a new month	Bills are calculated from area and rate; existing period bills are skipped	PASS
T4	Record a partial payment	Bill status changes to PARTIAL	PASS
T5	Attempt to pay more than the remaining balance	Overpayment is rejected	PASS
T6	Pay the remaining balance	Bill status changes to PAID	PASS
T7	Submit a complaint	Complaint is stored with resident reference	PASS

5.1 Results

- Flat and resident records can be created through the local interface.
- Monthly billing uses stored flat area and a user-supplied sample rate.
- The database can represent multiple payment transactions against one bill.
- Payment totals drive the unpaid/partial/paid status, and overpayment attempts are blocked.
- Complaint entries and due/collection reports are available through separate pages.

5.2 Test limitations

These tests are a basic prototype smoke test, not a full security, load or production-readiness audit. Before final evaluation, the team should manually demonstrate the forms in a browser, check the exact MySQL schema on the lab's MySQL installation if required, and capture final screenshots after entering clean demo records.

6. Conclusion and Future Scope

The Residential Society Billing System demonstrates how a relational database can support a society office by organizing flats, residents, recurring bills, payment transactions and complaints. The design uses primary keys and foreign keys to connect the entities and a unique constraint to prevent duplicate bills for the same flat and month. The local prototype brings the main workflows into one browser interface and supports payment-status tracking and due reports.

The project also applies classroom DBMS concepts to a practical context, including ER modelling, schema design, data integrity, joins, aggregation and trigger-based status calculation. The current submission is an academic prototype; it should not be used with real resident records or public-facing production traffic.

6.1 Future enhancements

- Role-based authentication for administrators and office staff.
- A fully configured MySQL-backed application with migrations and database credentials stored securely.
- PDF receipts, printable bills and exportable reports.
- Automated due-date reminders, backup and audit history.
- Resident portal, online payment-gateway integration and mobile-friendly access.
- Additional modules such as visitor logs, amenities and staff payroll, as noted in the review presentation.

References

1. Project source material: “DBMS Project - Design and Implementation of a Database Management System for a Residential Society: Resident & Maintenance Billing System”, Review 1 presentation provided by the project team.

2. Python Standard Library documentation: https://docs.python.org/3/library/

3. SQLite documentation: https://www.sqlite.org/docs.html

4. MySQL Reference Manual (for the included MySQL 8+ schema and triggers): https://dev.mysql.com/doc/

Before printing: fill the supervisor/university placeholders on the cover page, verify all faculty-specific formatting rules, and include screenshots or appendices if required.